Foundations of Cross-validation statistics

[Cross-validation statistics]

1.0 Content Block

L λ i = Relative Accuracy i + ∑ c = 1 C γ c 1 − γ c Relative Simplicity i , c . {\displaystyle L_{\lambda _{i}}={\mbox{ Relative Accuracy}}_{i}+\sum _{c=1}^{C}{\frac {\gamma _{c}}{1-\gamma _{c}}}{\mbox{ Relative Simplicity}}_{i,c}.}

2.0 Content Block

Using prior information When users apply cross-validation to select a good configuration λ {\displaystyle \lambda } , then they might want to balance the cross-validated choice with their own estimate of the configuration. In this way, they can attempt to counter the volatility of cross-validation when the sample size is small and include relevant information from previous research. In a forecasting combination exercise, for instance, cross-validation can be applied to estimate the weights that are assigned to each forecast. Since a simple equal-weighted forecast is difficult to beat, a penalty can be added for deviating from equal weights. Or, if cross-validation is applied to assign individual weights to observations, then one can penalize deviations from equal weights to avoid wasting potentially relevant information. Hoornweg (2018) shows how a tuning parameter γ {\displaystyle \gamma } can be defined so that a user can intuitively balance between the accuracy of cross-validation and the simplicity of sticking to a reference parameter λ R {\displaystyle \lambda _{R}} that is defined by the user. If λ i {\displaystyle \lambda _{i}} denotes the i t h {\displaystyle i^{th}} candidate configuration that might be selected, then the loss function that is to be minimized can be defined as

3.0 Content Block

If the model is correctly specified, it can be shown under mild assumptions that the expected value of the MSE for the training set is $(n - p - 1)/(n + p + 1) < 1$ times the expected value of the MSE for the validation set (the expected value is taken over the distribution of training sets). Thus, a fitted model and computed MSE on the training set will result in an optimistically biased assessment of how well the model will fit an independent data set. This biased estimate is called the in-sample estimate of the fit, whereas the cross-validation estimate is an out-of-sample estimate. Since in linear regression it is possible to directly compute the factor $(n - p - 1)/(n + p + 1)$ by which the training MSE underestimates the validation MSE under the assumption that the model specification is valid, cross-validation can be used for checking whether the model has been overfitted, in which case the MSE in the validation set will substantially exceed its anticipated value. (Cross-validation in the context of linear regression is also useful in that it can be used to select an optimally regularized cost function.)

4.0 Content Block

Leave-p-out cross-validation Leave-p-out cross-validation (LpO CV) involves using p observations as the validation set and the remaining observations as the training set. This is repeated on all ways to cut the original sample on a validation set of p observations and a training set. LpO cross-validation require training and validating the model C p n {\displaystyle C_{p}^{n}} times, where n is the number of observations in the original sample, and where C p n {\displaystyle C_{p}^{n}} is the binomial coefficient. For p > 1 and for even moderately large n, LpO CV can become computationally infeasible. For example, with n = 100 and p = 30, C 30 100 ≈ 3 × 10 25 . {\displaystyle C_{30}^{100}\approx 3\times 10^{25}.}

5.0 Content Block

k-fold cross-validation with validation and test set This is a type of k*l-fold cross-validation when l = k - 1. A single k-fold cross-validation is used with both a validation and test set. The total data set is split into k sets. One by one, a set is selected as test set. Then, one by one, one of the remaining sets is used as a validation set and the other k - 2 sets are used as training sets until all possible combinations have been evaluated. Similar to the k*l-fold cross validation, the training set is used for model fitting and the validation set is used for model evaluation for each of the hyperparameter sets. Finally, for the selected parameter set, the test set is used to evaluate the model with the best parameter set. Here, two variants are possible: either evaluating the model that was trained on the training set or evaluating a new model that was fit on the combination of the training and the validation set.

6.0 Content Block

Motivation Assume a model with one or more unknown parameters, and a data set to which the model can be fit (the training data set). The fitting process optimizes the model parameters to make the model fit the training data as well as possible. If an independent sample of validation data is taken from the same population as the training data, it will generally turn out that the model does not fit the validation data as well as it fits the training data. The size of this difference is likely to be large especially when the size of the training data set is small, or when the number of parameters in the model is large. Cross-validation is a way to estimate the size of this effect.

7.0 Content Block

k-fold cross-validation In k-fold cross-validation, the original sample is randomly partitioned into k equal sized subsamples, often referred to as "folds". Of the k subsamples, a single subsample is retained as the validation data for testing the model, and the remaining k – 1 subsamples are used as training data. The cross-validation process is then repeated k times, with each of the k subsamples used exactly once as the validation data. The k results can then be averaged to produce a single estimation. The advantage of this method over repeated random sub-sampling (see below) is that all observations are used for both training and validation, and each observation is used for validation exactly once. 10-fold cross-validation is commonly used, but in general k remains an unfixed parameter. For example, setting k = 2 results in 2-fold cross-validation. In 2-fold cross-validation, the dataset is randomly shuffled into two sets d0 and d1, so that both sets are equal size (this is usually implemented by shuffling the data array and then splitting it in two). We then train on d0 and validate on d1, followed by training on d1 and validating on d0. When k = n (the number of observations), k-fold cross-validation is equivalent to leave-one-out cross-validation. In stratified k-fold cross-validation, the partitions are selected so that the mean response value is approximately equal in all the partitions. In the case of binary classification, this means that each partition contains roughly the same proportions of the two types of class labels. In repeated cross-validation the data is randomly split into k partitions several times. The performance of the model can thereby be averaged over several runs, but this is rarely desirable in practice. When many different statistical or machine learning models are being considered, greedy k-fold cross-validation can be used to quickly identify the most promising candidate models.